

Customer Segmentation

```
In [1]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.decomposition import PCA
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (
    silhouette_score,
    calinski_harabasz_score,
    davies_bouldin_score,
)
from scipy.cluster.hierarchy import dendrogram, linkage

sns.set_style("whitegrid")
RANDOM_STATE = 42
```

2. Load data and first look

```
In [2]: df = pd.read_csv("customer_data.csv")
df.shape
```

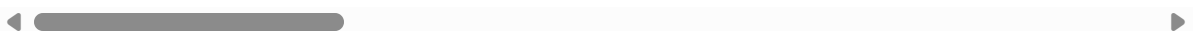
Out[2]: (10000, 42)

```
In [3]: df.head()
```

```
Out[3]:
```

	Customer_ID	Birth_Year	Education_Level	Marital_Status	Annual_Income	Kids_Home	T
0	1	1978	Graduation	Widow	40175.23	1	
1	2	1991	PhD	Widow	27979.68	0	
2	3	1968	PhD	Married	64747.68	0	
3	4	1954	Graduation	Widow	50185.85	2	
4	5	1982	Graduation	Married	53163.09	1	

5 rows × 42 columns



```
In [4]: df.columns.tolist()
```

```
Out[4]: ['Customer_ID',  
        'Birth_Year',  
        'Education_Level',  
        'Marital_Status',  
        'Annual_Income',  
        'Kids_Home',  
        'Teens_Home',  
        'Customer_Since',  
        'Last_Purchase_Recency',  
        'Spent_Wine',  
        'Spent_Fruits',  
        'Spent_Meat',  
        'Spent_Fish',  
        'Spent_Sweets',  
        'Spent_Gold',  
        'Deals_Purchased',  
        'Web_Purchases',  
        'Catalog_Purchases',  
        'Store_Purchases',  
        'Monthly_Web_Visits',  
        'Campaign1_Accepted',  
        'Campaign2_Accepted',  
        'Campaign3_Accepted',  
        'Campaign4_Accepted',  
        'Campaign5_Accepted',  
        'Filed_Complaint',  
        'Contact_Cost',  
        'Revenue_Contribution',  
        'Campaign_Response',  
        'Country',  
        'City',  
        'Region_Code',  
        'Mobile_App_User',  
        'Email_Subscriber',  
        'Device_Type',  
        'Referred_By_Friend',  
        'Social_Media_User',  
        'Membership_Tier',  
        'Total_Amount_Spent',  
        'Customer_Age',  
        'Family_Size',  
        'Senior_Citizen']
```

```
In [5]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 42 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer_ID                          10000 non-null  int64
1   Birth_Year                          10000 non-null  int64
2   Education_Level                     10000 non-null  str
3   Marital_Status                     10000 non-null  str
4   Annual_Income                      10000 non-null  float64
5   Kids_Home                          10000 non-null  int64
6   Teens_Home                         10000 non-null  int64
7   Customer_Since                     10000 non-null  str
8   Last_Purchase_Recency              10000 non-null  int64
9   Spent_Wine                         10000 non-null  int64
10  Spent_Fruits                       10000 non-null  int64
11  Spent_Meat                         10000 non-null  int64
12  Spent_Fish                        10000 non-null  int64
13  Spent_Sweets                      10000 non-null  int64
14  Spent_Gold                        10000 non-null  int64
15  Deals_Purchased                   10000 non-null  int64
16  Web_Purchases                     10000 non-null  int64
17  Catalog_Purchases                 10000 non-null  int64
18  Store_Purchases                   10000 non-null  int64
19  Monthly_Web_Visits                10000 non-null  int64
20  Campaign1_Accepted                10000 non-null  int64
21  Campaign2_Accepted                10000 non-null  int64
22  Campaign3_Accepted                10000 non-null  int64
23  Campaign4_Accepted                10000 non-null  int64
24  Campaign5_Accepted                10000 non-null  int64
25  Filed_Complaint                   10000 non-null  int64
26  Contact_Cost                      10000 non-null  int64
27  Revenue_Contribution              10000 non-null  int64
28  Campaign_Response                 10000 non-null  int64
29  Country                           10000 non-null  str
30  City                              10000 non-null  str
31  Region_Code                       10000 non-null  int64
32  Mobile_App_User                   10000 non-null  int64
33  Email_Subscriber                  10000 non-null  int64
34  Device_Type                       10000 non-null  str
35  Referred_By_Friend                10000 non-null  int64
36  Social_Media_User                 10000 non-null  int64
37  Membership_Tier                   10000 non-null  str
38  Total_Amount_Spent                10000 non-null  int64
39  Customer_Age                     10000 non-null  int64
40  Family_Size                       10000 non-null  int64
41  Senior_Citizen                    10000 non-null  int64
dtypes: float64(1), int64(34), str(7)
memory usage: 3.2 MB

```

```
In [6]: df.describe().T
```

Out[6]:

	count	mean	std	min	25%	50%
Customer_ID	10000.0	5000.500000	2886.895680	1.00	2500.750	5000.500
Birth_Year	10000.0	1970.798400	17.903223	1940.00	1956.000	1971.000
Annual_Income	10000.0	49934.047232	14870.373127	-7845.63	39607.725	49966.965
Kids_Home	10000.0	0.997300	0.818388	0.00	0.000	1.000
Teens_Home	10000.0	0.989300	0.814894	0.00	0.000	1.000
Last_Purchase_Recency	10000.0	49.592500	29.101686	0.00	25.000	49.000
Spent_Wine	10000.0	497.928400	287.932884	0.00	245.000	500.000
Spent_Fruits	10000.0	49.709000	28.852961	0.00	25.000	49.000
Spent_Meat	10000.0	401.292100	230.822103	0.00	202.000	406.000
Spent_Fish	10000.0	148.804400	86.715667	0.00	73.000	148.000
Spent_Sweets	10000.0	99.322100	57.374302	0.00	50.000	99.000
Spent_Gold	10000.0	250.722100	143.771641	0.00	126.000	251.000
Deals_Purchased	10000.0	4.511500	2.877098	0.00	2.000	5.000
Web_Purchases	10000.0	4.482300	2.879638	0.00	2.000	4.000
Catalog_Purchases	10000.0	4.456100	2.884043	0.00	2.000	4.000
Store_Purchases	10000.0	4.520300	2.853489	0.00	2.000	5.000
Monthly_Web_Visits	10000.0	9.470800	5.793643	0.00	4.000	9.000
Campaign1_Accepted	10000.0	0.498600	0.500023	0.00	0.000	0.000
Campaign2_Accepted	10000.0	0.508900	0.499946	0.00	0.000	1.000
Campaign3_Accepted	10000.0	0.515200	0.499794	0.00	0.000	1.000
Campaign4_Accepted	10000.0	0.499700	0.500025	0.00	0.000	0.000
Campaign5_Accepted	10000.0	0.492800	0.499973	0.00	0.000	0.000
Filed_Complaint	10000.0	0.498200	0.500022	0.00	0.000	0.000
Contact_Cost	10000.0	3.000000	0.000000	3.00	3.000	3.000
Revenue_Contribution	10000.0	11.000000	0.000000	11.00	11.000	11.000
Campaign_Response	10000.0	0.502500	0.500019	0.00	0.000	1.000
Region_Code	10000.0	551.009400	258.756758	100.00	327.000	550.500
Mobile_App_User	10000.0	0.494000	0.499989	0.00	0.000	0.000
Email_Subscriber	10000.0	0.511400	0.499895	0.00	0.000	1.000
Referred_By_Friend	10000.0	0.504200	0.500007	0.00	0.000	1.000

	count	mean	std	min	25%	50%
Social_Media_User	10000.0	0.500600	0.500025	0.00	0.000	1.000
Total_Amount_Spent	10000.0	1447.778100	408.159165	210.00	1158.750	1447.000
Customer_Age	10000.0	54.201600	17.903223	24.00	39.000	54.000
Family_Size	10000.0	2.375000	1.248812	0.00	1.000	2.000
Senior_Citizen	10000.0	0.400700	0.490065	0.00	0.000	0.000

Data quality checks

```
In [7]: print("Total null values   :", df.isnull().sum().sum())
        print("Duplicate rows       :", df.duplicated().sum())
        print("Duplicate Customer_ID:", df["Customer_ID"].duplicated().sum())
```

```
Total null values   : 0
Duplicate rows       : 0
Duplicate Customer_ID: 0
```

```
In [8]: numeric_cols = df.select_dtypes(include="number").columns.tolist()
        categorical_cols = df.select_dtypes(exclude="number").columns.tolist()
        print("Numeric columns   :", len(numeric_cols))
        print("Categorical columns:", categorical_cols)
```

```
Numeric columns   : 35
Categorical columns: ['Education_Level', 'Marital_Status', 'Customer_Since', 'Country', 'City', 'Device_Type', 'Membership_Tier']
```

```
In [9]: for col in ["Education_Level", "Marital_Status", "Country", "City", "Device_Type",
                    print(f"\n--- {col} ---")
                    print(df[col].value_counts(dropna=False))
```

```

--- Education_Level ---
Education_Level
Graduation    5027
Master        2026
Basic         1976
PhD           971
Name: count, dtype: int64

```

```

--- Marital_Status ---
Marital_Status
Widow         2079
Divorced      2061
Single        1976
Married       1947
Together      1937
Name: count, dtype: int64

```

```

--- Country ---
Country
Brazil        2102
Germany       2047
India         1960
UK            1956
USA           1935
Name: count, dtype: int64

```

```

--- City ---
City
New York      2097
London        1998
São Paulo     1986
Berlin        1985
Mumbai        1934
Name: count, dtype: int64

```

```

--- Device_Type ---
Device_Type
Mobile        4926
Desktop       4091
Tablet        983
Name: count, dtype: int64

```

```

--- Membership_Tier ---
Membership_Tier
Basic         4003
Silver        2978
Gold          2006
Platinum      1013
Name: count, dtype: int64

```

```

In [10]: # Negative values in numeric columns
neg = (df[numeric_cols] < 0).sum()
print(neg[neg > 0])

```

```

Annual_Income    5
dtype: int64

```

```
In [11]: nunique = df.nunique().sort_values()
print(nunique.head(12))
print("\nConstant columns:", nunique[nunique == 1].index.tolist())
```

```
Revenue_Contribution    1
Contact_Cost            1
Campaign1_Accepted      2
Campaign2_Accepted      2
Campaign3_Accepted      2
Campaign4_Accepted      2
Filed_Complaint         2
Campaign5_Accepted      2
Campaign_Response       2
Referred_By_Friend      2
Email_Subscriber        2
Senior_Citizen          2
dtype: int64
```

Constant columns: ['Revenue_Contribution', 'Contact_Cost']

```
In [12]: # Consistency checks between related columns
spent_cols = ["Spent_Wine", "Spent_Fruits", "Spent_Meat", "Spent_Fish", "Spent_Sweet"]
print("Total_Amount_Spent == sum of Spent_* columns :", (df[spent_cols].sum(axis=1) == df["Total_Amount_Spent"]))
print("Customer_Age == 2025 - Birth_Year :", ((2025 - df["Birth_Year"]) == df["Customer_Age"]))
print("Kids_Home + Teens_Home vs Family_Size corr :", round(df[["Kids_Home", "Teens_Home", "Family_Size"]].corr().iloc[0,2]))
print("\nCountry vs City (a customer in Germany can be in Mumbai -> City is not tied to Country):")
print(pd.crosstab(df["Country"], df["City"]))
```

```
Total_Amount_Spent == sum of Spent_* columns : True
Customer_Age == 2025 - Birth_Year : True
Kids_Home + Teens_Home vs Family_Size corr : 0.92
```

Country vs City (a customer in Germany can be in Mumbai -> City is not tied to Country):

City	Berlin	London	Mumbai	New York	São Paulo
Brazil	434	411	401	427	429
Germany	418	396	383	441	409
India	382	407	387	422	362
UK	394	399	364	421	378
USA	357	385	399	386	408

Findings from inspection

- No missing values and no duplicate rows / IDs - the dataset is cleaner than the old churn file.
- `Annual_Income` contains **negative values** (invalid) - same kind of issue as in the old notebook.
- `Contact_Cost` and `Revenue_Contribution` are **constants** (3 and 11) -> useless for clustering.
- `Customer_Since` is a text date (dd-mm-yyyy) -> convert to datetime and create a *tenure* feature.

- `Birth_Year` duplicates `Customer_Age` (age = 2025 – birth year);
`Total_Amount_Spent` is exactly the sum of the six `Spent_*` columns;
`Senior_Citizen` is derived from age.
- `City` is not consistent with `Country`, and `Region_Code` has ~900 unique codes (an ID-like column).

Data cleaning

```
In [13]: neg_income = (df["Annual_Income"] < 0).sum()
median_income = df.loc[df["Annual_Income"] >= 0, "Annual_Income"].median()
df.loc[df["Annual_Income"] < 0, "Annual_Income"] = np.nan
df["Annual_Income"] = df["Annual_Income"].fillna(median_income)
print(f"Negative incomes fixed: {neg_income} | median used: {median_income:.2f}")
print("Negative values left :", (df.select_dtypes(include='number') < 0).sum().sum())
```

Negative incomes fixed: 5 | median used: 49975.81
Negative values left : 0

```
In [14]: for col in ["Education_Level", "Marital_Status", "Country", "City", "Device_Type",
df[col] = df[col].str.strip()
df[["Education_Level", "Marital_Status", "Country", "City", "Device_Type", "Members
```

```
Out[14]: Education_Level    4
Marital_Status            5
Country                  5
City                     5
Device_Type              3
Membership_Tier          4
dtype: int64
```

```
In [15]: df["Customer_Since"] = pd.to_datetime(df["Customer_Since"], format="%d-%m-%Y")
reference_date = df["Customer_Since"].max()
df["Tenure_Years"] = ((reference_date - df["Customer_Since"]).dt.days / 365.25).rou
print("Reference date:", reference_date.date())
df[["Customer_Since", "Tenure_Years"]].head()
```

Reference date: 2015-12-31

```
Out[15]:
```

	Customer_Since	Tenure_Years
0	2010-09-26	5.26
1	2012-11-03	3.16
2	2013-10-31	2.17
3	2010-04-30	5.67
4	2015-09-25	0.27

```
In [16]: df["Total_Children"] = df["Kids_Home"] + df["Teens_Home"]
df["Total_Purchases"] = df[["Web_Purchases", "Catalog_Purchases", "Store_Purchas
df["Campaigns_Accepted"] = df[[f"Campaign{i}_Accepted" for i in range(1, 6)]].sum(a
df[["Total_Children", "Total_Purchases", "Campaigns_Accepted"]].describe().T
```

Out[16]:

	count	mean	std	min	25%	50%	75%	max
Total_Children	10000.0	1.9866	1.150892	0.0	1.0	2.0	3.0	4.0
Total_Purchases	10000.0	13.4587	4.959290	0.0	10.0	13.0	17.0	27.0
Campaigns_Accepted	10000.0	2.5152	1.125031	0.0	2.0	3.0	3.0	5.0

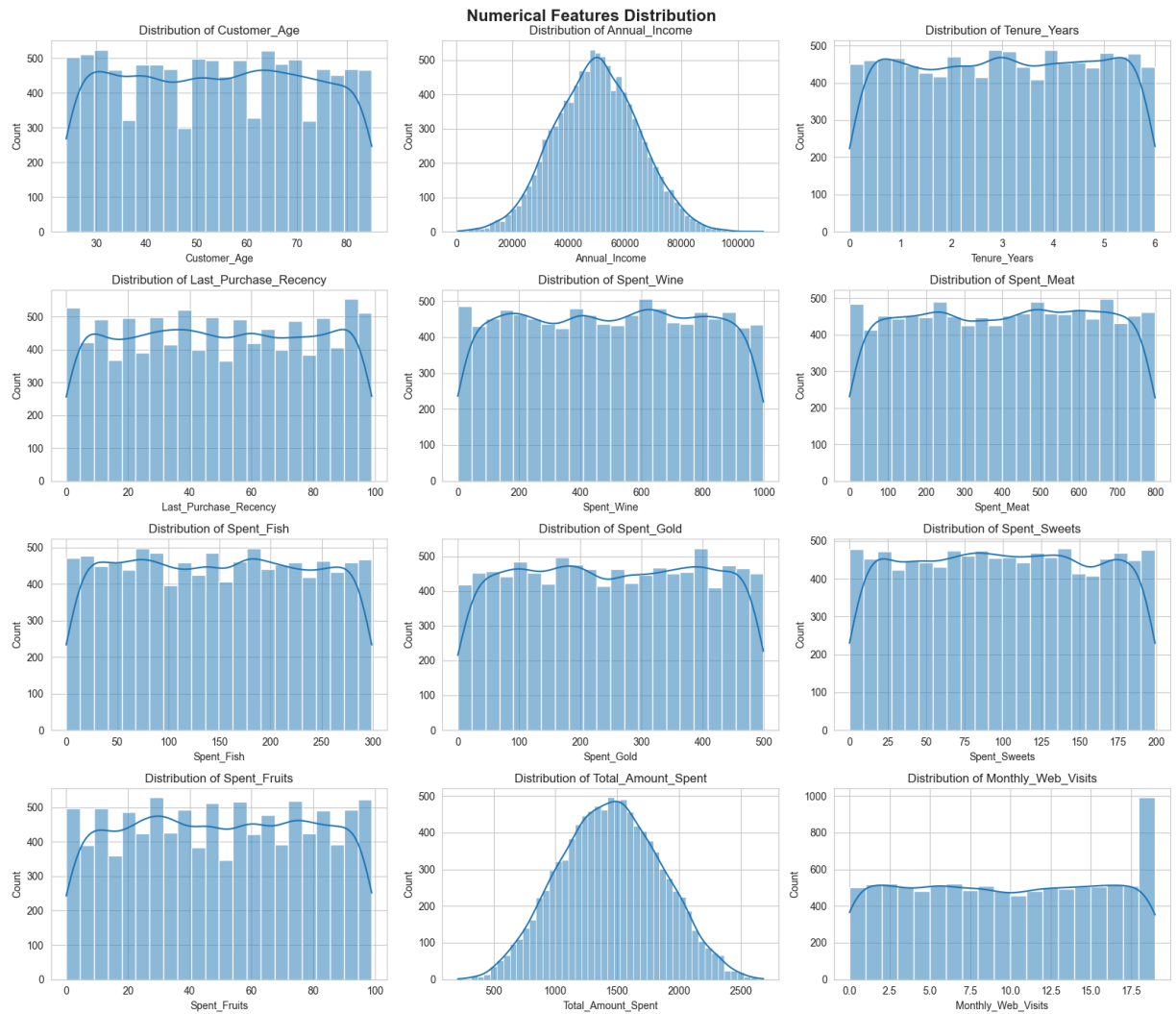
```
In [17]: print("Nulls      :", df.isnull().sum().sum())
print("Duplicates:", df.duplicated().sum())
df.to_csv("cleaned_customer_data.csv", index=False)
print("Saved cleaned_customer_data.csv", df.shape)
```

```
Nulls      : 0
Duplicates: 0
Saved cleaned_customer_data.csv (10000, 46)
```

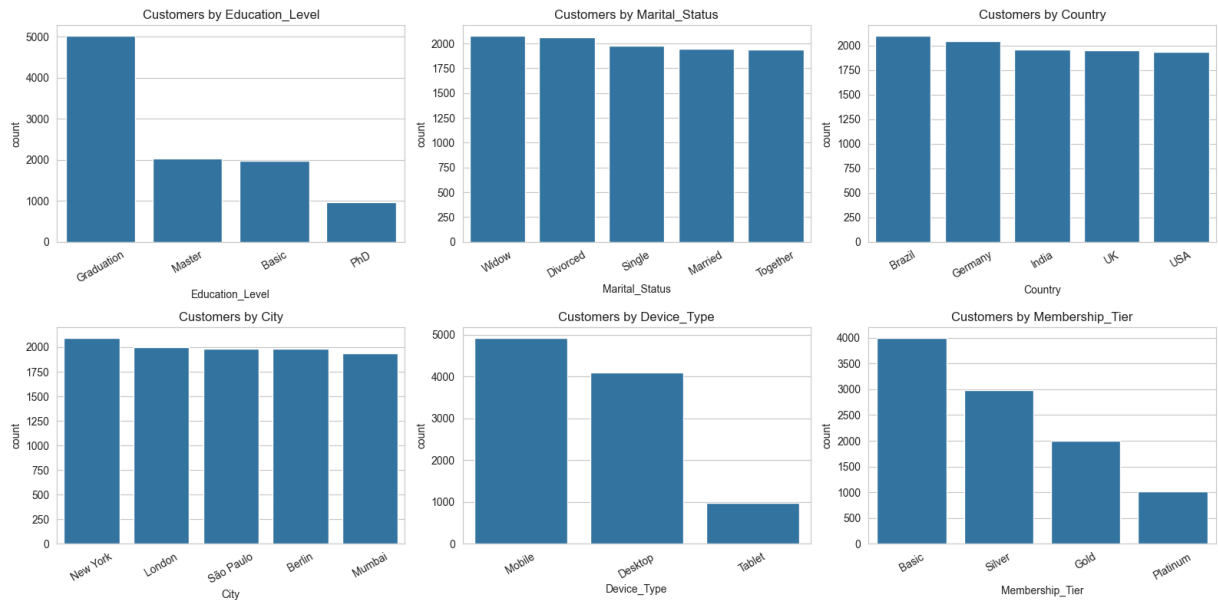
Exploratory data analysis

```
In [18]: eda_cols = ["Customer_Age", "Annual_Income", "Tenure_Years", "Last_Purchase_Recency",
                    "Spent_Wine", "Spent_Meat", "Spent_Fish", "Spent_Gold", "Spent_Sweets",
                    "Spent_Fruits", "Total_Amount_Spent", "Monthly_Web_Visits"]

fig, axes = plt.subplots(4, 3, figsize=(16, 14))
for ax, col in zip(axes.flatten(), eda_cols):
    sns.histplot(df[col], kde=True, ax=ax)
    ax.set_title(f"Distribution of {col}")
plt.suptitle("Numerical Features Distribution", fontsize=16, fontweight="bold")
plt.tight_layout()
plt.show()
```

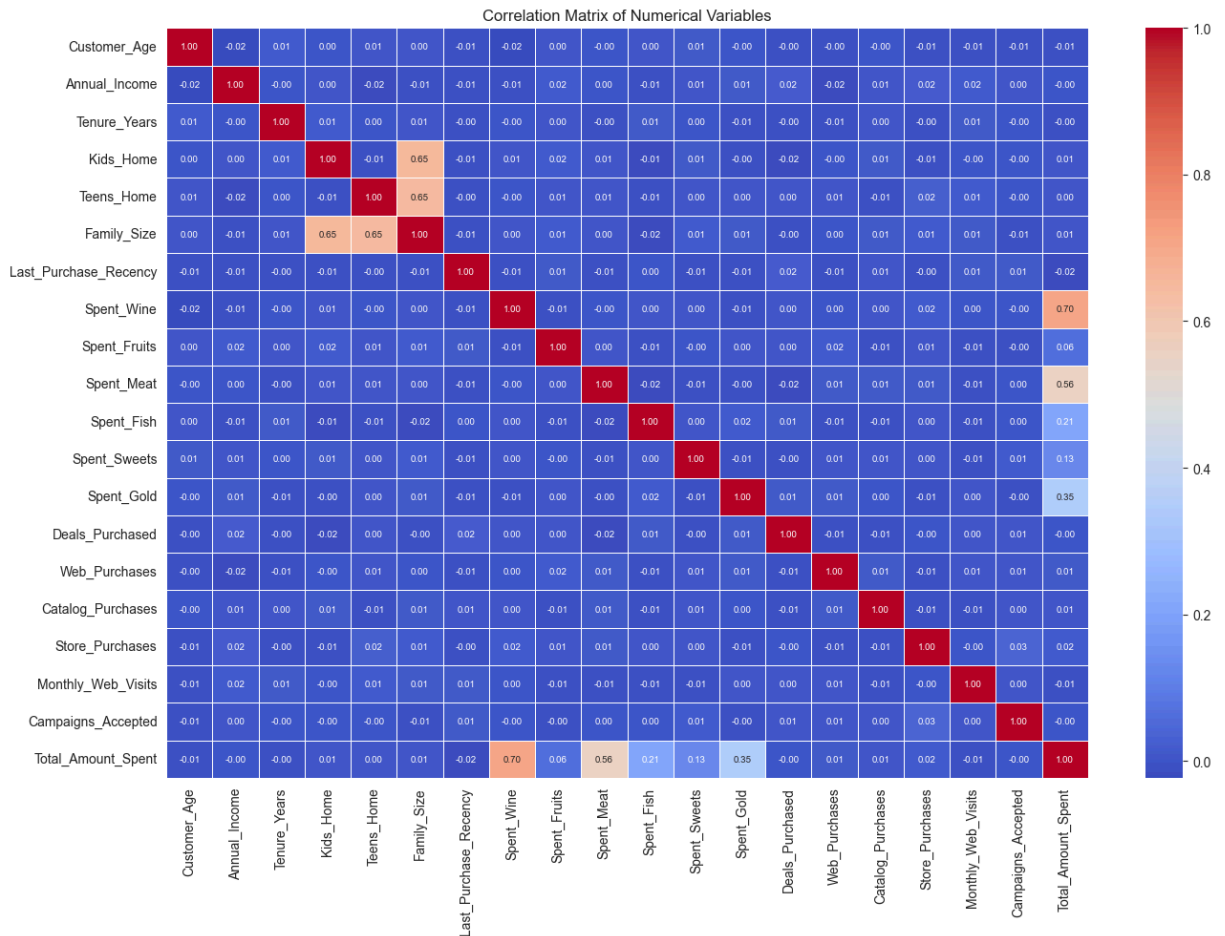


```
In [19]: fig, axes = plt.subplots(2, 3, figsize=(16, 8))
for ax, col in zip(axes.flatten(), ["Education_Level", "Marital_Status", "Country",
order = df[col].value_counts().index
sns.countplot(data=df, x=col, order=order, ax=ax)
ax.set_title(f"Customers by {col}")
ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()
```



```
In [20]: corr_cols = ["Customer_Age", "Annual_Income", "Tenure_Years", "Kids_Home", "Teens_H",
    "Last_Purchase_Recency", "Spent_Wine", "Spent_Fruits", "Spent_Meat", "
    "Spent_Sweets", "Spent_Gold", "Deals_Purchased", "Web_Purchases", "Cat
    "Store_Purchases", "Monthly_Web_Visits", "Campaigns_Accepted", "Total_
correlation = df[corr_cols].corr()

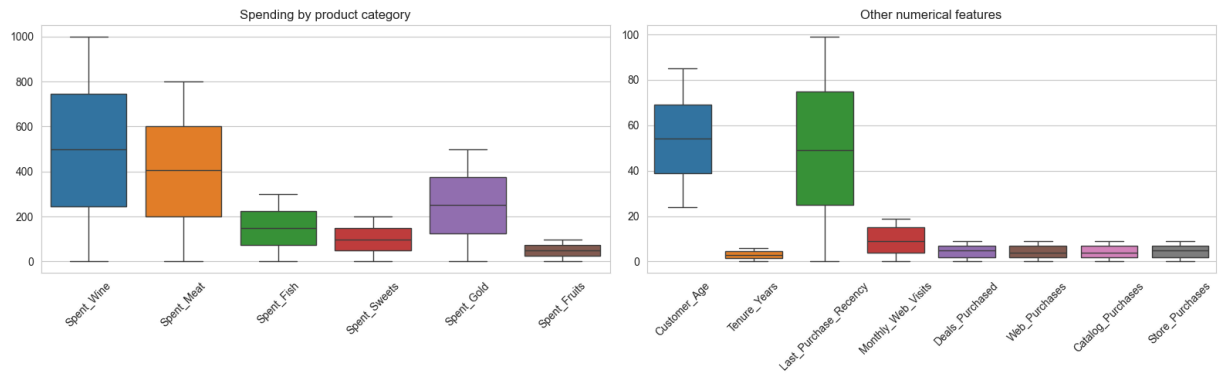
plt.figure(figsize=(14, 10))
sns.heatmap(correlation, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5, an
plt.title("Correlation Matrix of Numerical Variables")
plt.tight_layout()
plt.show()
```



```
In [21]: # Strongest pairwise correlations
pairs = correlation.abs().where(np.triu(np.ones(correlation.shape), k=1).astype(bool))
pairs.head(8)
```

```
Out[21]: Spent_Wine      Total_Amount_Spent      0.701638
Kids_Home      Family_Size      0.651928
Teens_Home     Family_Size      0.645584
Spent_Meat     Total_Amount_Spent  0.557522
Spent_Gold     Total_Amount_Spent  0.345430
Spent_Fish     Total_Amount_Spent  0.212317
Spent_Sweets   Total_Amount_Spent  0.130721
Spent_Fruits   Total_Amount_Spent  0.064872
dtype: float64
```

```
In [22]: # Box plots (spend columns share a scale, so they are plotted together)
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
sns.boxplot(data=df[["Spent_Wine", "Spent_Meat", "Spent_Fish", "Spent_Sweets", "Spent_Gold"]],
            axes[0].set_title("Spending by product category"))
sns.boxplot(data=df[["Customer_Age", "Tenure_Years", "Last_Purchase_Recency", "Monthly_Web_Visits",
                    "Deals_Purchased", "Web_Purchases", "Catalog_Purchases", "Store_Purchases"]],
            axes[1].set_title("Other numerical features"))
for ax in axes: ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```



```
In [23]: # IQR outlier check (same method as the original notebook)
outlier_cols = ["Customer_Age", "Annual_Income", "Tenure_Years", "Last_Purchase_Rec",
                "Spent_Wine", "Spent_Fruits", "Spent_Meat", "Spent_Fish", "Spent_Sw",
                "Deals_Purchased", "Web_Purchases", "Catalog_Purchases", "Store_Pur"]
rows, total_outliers, total_values = [], 0, 0
for col in outlier_cols:
    Q1, Q3 = df[col].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    n_out = int(((df[col] < lower) | (df[col] > upper)).sum())
    rows.append({"Column": col, "Lower": round(lower, 2), "Upper": round(upper, 2),
                "Outliers": n_out, "Outlier_%": round(n_out / len(df) * 100, 2)})
    total_outliers += n_out
    total_values += len(df)

display(pd.DataFrame(rows))
print(f"Overall outlier percentage: {total_outliers / total_values * 100:.2f}%")
```

	Column	Lower	Upper	Outliers	Outlier_%
0	Customer_Age	-6.00	114.00	0	0.00
1	Annual_Income	9019.78	90682.45	52	0.52
2	Tenure_Years	-3.06	9.10	0	0.00
3	Last_Purchase_Recency	-50.00	150.00	0	0.00
4	Spent_Wine	-508.00	1500.00	0	0.00
5	Spent_Fruits	-50.00	150.00	0	0.00
6	Spent_Meat	-395.00	1197.00	0	0.00
7	Spent_Fish	-152.00	448.00	0	0.00
8	Spent_Sweets	-97.00	295.00	0	0.00
9	Spent_Gold	-249.00	751.00	0	0.00
10	Deals_Purchased	-5.50	14.50	0	0.00
11	Web_Purchases	-5.50	14.50	0	0.00
12	Catalog_Purchases	-5.50	14.50	0	0.00
13	Store_Purchases	-5.50	14.50	0	0.00
14	Monthly_Web_Visits	-12.50	31.50	0	0.00

Overall outlier percentage: 0.03%

Preprocessing for unsupervised learning

```
In [24]: drop_cols = ["Customer_ID", "Campaign_Response", "Contact_Cost", "Revenue_Contribut
            "Total_Amount_Spent", "Senior_Citizen", "Customer_Since", "Region_Code
            "Total_Children", "Total_Purchases", "Campaigns_Accepted"]
```

```
df_cluster = df.drop(columns=drop_cols)
```

```
numeric_columns = df_cluster.select_dtypes(include="number").columns.tolist()
categorical_columns = df_cluster.select_dtypes(exclude="number").columns.tolist()
print("Numeric      :", len(numeric_columns))
print("Categorical:", categorical_columns)
print("Nulls       :", df_cluster.isnull().sum().sum())
```

Numeric : 28

Categorical: ['Education_Level', 'Marital_Status', 'Country', 'Device_Type', 'Member
ship_Tier']

Nulls : 0

Convert categorical features to numerical (one-hot encoding)

```
In [25]: df_encoded = pd.get_dummies(df_cluster, columns=categorical_columns, drop_first=False)
print(df_encoded.shape)
```

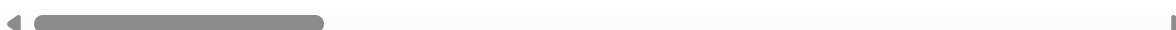
```
df_encoded.head()
```

```
(10000, 49)
```

```
Out[25]:
```

	Annual_Income	Kids_Home	Teens_Home	Last_Purchase_Recency	Spent_Wine	Spent_Fri
0	40175.23	1	1	68	440	
1	27979.68	0	1	99	826	
2	64747.68	0	0	63	895	
3	50185.85	2	2	65	127	
4	53163.09	1	2	49	140	

```
5 rows × 49 columns
```



Feature scaling (StandardScaler)

```
In [26]: X = df_encoded.copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Feature matrix:", X_scaled.shape)
```

```
Feature matrix: (10000, 49)
```

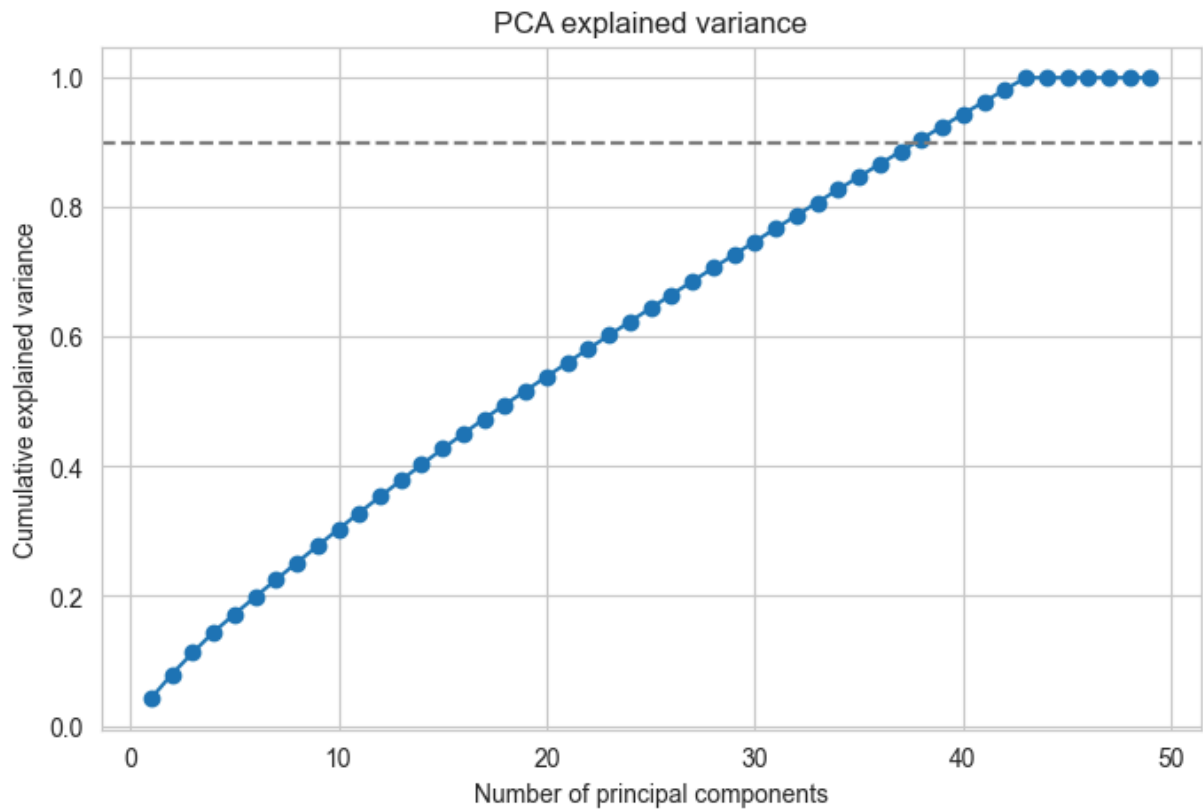
```
In [27]: # PCA is used only for 2-D visualisation of the clusters (the models run on the full data)
pca_full = PCA().fit(X_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
print("Variance explained by PC1 + PC2:", round(cum_var[1] * 100, 2), "%")
print("Components needed for 90% variance:", int(np.argmax(cum_var >= 0.90)) + 1)

pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(cum_var) + 1), cum_var, marker="o")
plt.axhline(0.90, ls="--", color="grey")
plt.xlabel("Number of principal components"); plt.ylabel("Cumulative explained variance")
plt.title("PCA explained variance")
plt.show()
```

```
Variance explained by PC1 + PC2: 7.92 %
```

```
Components needed for 90% variance: 38
```



K-Means clustering

Choosing K - elbow method and silhouette score

```
In [28]: k_values = range(2, 11)
inertia_values, silhouette_values = [], []

for k in k_values:
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertia_values.append(km.inertia_)
    silhouette_values.append(silhouette_score(X_scaled, labels))

results = pd.DataFrame({"K": list(k_values), "Inertia": inertia_values, "Silhouette": silhouette_values})
```

```

File "c:\Users\vanda\AppData\Local\Programs\Python\Python313\Lib\site-packages\job
lib\externals\loky\backend\context.py", line 257, in _count_physical_cores
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
File "c:\Users\vanda\AppData\Local\Programs\Python\Python313\Lib\subprocess.py", 1
ine 554, in run
    with Popen(*popenargs, **kwargs) as process:
        ~~~~~^~~~~~
File "c:\Users\vanda\AppData\Local\Programs\Python\Python313\Lib\subprocess.py", 1
ine 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~^~~~~~
        pass_fds, cwd, env,
        ~~~~~
    ...<5 lines>...
        gid, gids, uid, umask,
        ~~~~~
        start_new_session, process_group)
        ~~~~~
File "c:\Users\vanda\AppData\Local\Programs\Python\Python313\Lib\subprocess.py", 1
ine 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    ~~~~~^~~~~~
        # no special security
        ~~~~~
    ...<4 lines>...
        cwd,
        ~~~~
        startupinfo)
        ~~~~~

```

Out[28]:

	K	Inertia	Silhouette_Score
0	2	472186.660593	0.036406
1	3	461493.511263	0.039713
2	4	450749.709189	0.051463
3	5	438308.287139	0.063142
4	6	431943.394920	0.060726
5	7	428878.455237	0.031286
6	8	425396.064536	0.044140
7	9	419787.157743	0.042979
8	10	417053.959716	0.038337

In [29]:

```

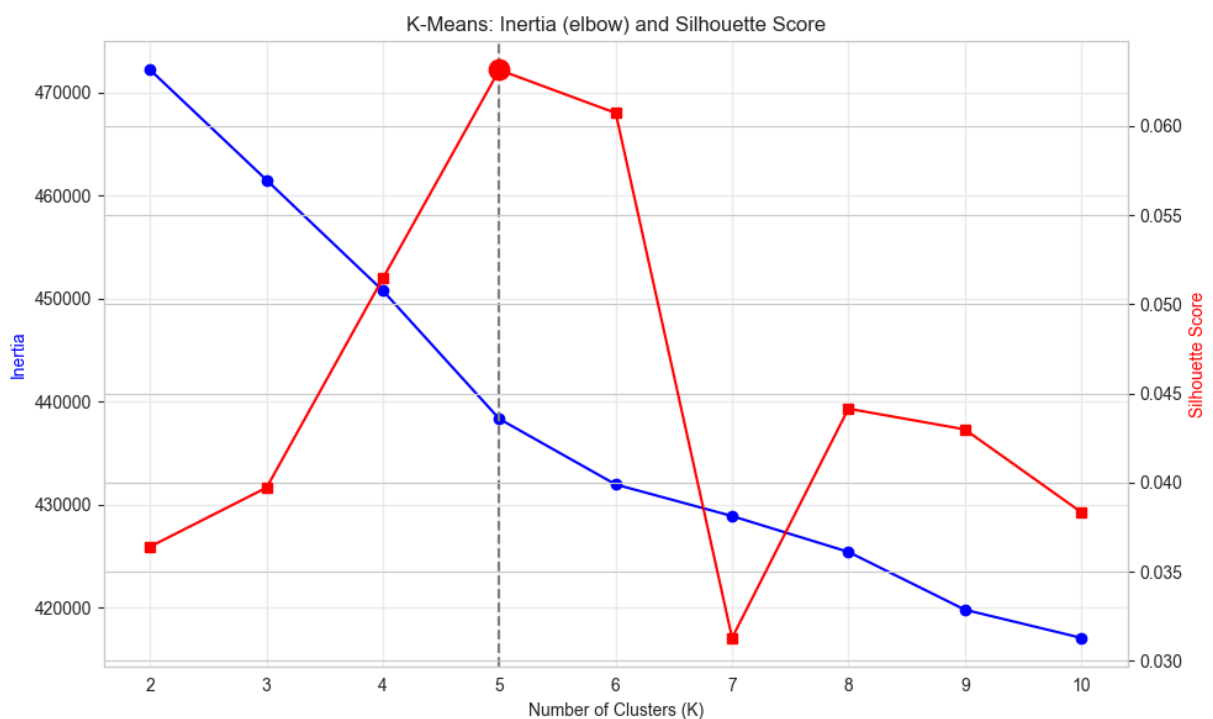
best_k = int(results.loc[results["Silhouette_Score"].idxmax(), "K"])
print("Best K based on Silhouette Score:", best_k)
print("Highest Silhouette Score      :", round(results["Silhouette_Score"].max(),

```

Best K based on Silhouette Score: 5
 Highest Silhouette Score : 0.0631

```
In [30]: fig, ax1 = plt.subplots(figsize=(10, 6))
ax1.plot(results["K"], results["Inertia"], marker="o", color="blue", label="Inertia")
ax1.set_xlabel("Number of Clusters (K)"); ax1.set_ylabel("Inertia", color="blue")
ax1.set_xticks(results["K"]); ax1.grid(True, alpha=0.3)

ax2 = ax1.twinx()
ax2.plot(results["K"], results["Silhouette_Score"], marker="s", color="red", label="Silhouette Score")
ax2.set_ylabel("Silhouette Score", color="red")
ax2.scatter(best_k, results["Silhouette_Score"].max(), color="red", s=140, zorder=5)
ax1.axvline(best_k, ls="--", color="grey")
plt.title("K-Means: Inertia (elbow) and Silhouette Score")
plt.tight_layout()
plt.show()
```



Final K-Means model

```
In [31]: final_kmeans = KMeans(n_clusters=best_k, random_state=RANDOM_STATE, n_init=10)
kmeans_labels = final_kmeans.fit_predict(X_scaled)

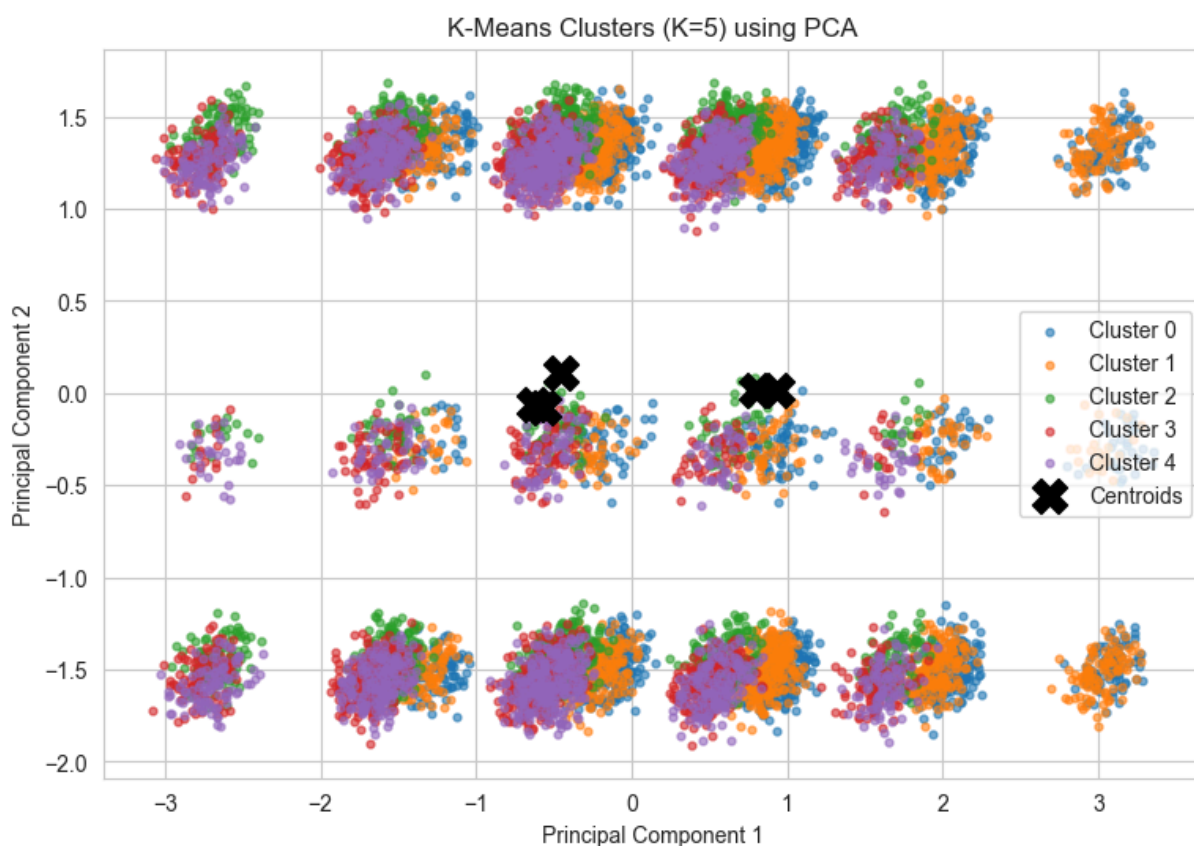
df_cluster = df.copy()
df_cluster["Cluster"] = kmeans_labels

print("Selected K      :", best_k)
print("Silhouette score :", round(silhouette_score(X_scaled, kmeans_labels), 4))
print("Inertia          :", round(final_kmeans.inertia_, 2))
print()
print(df_cluster["Cluster"].value_counts().sort_index())
```

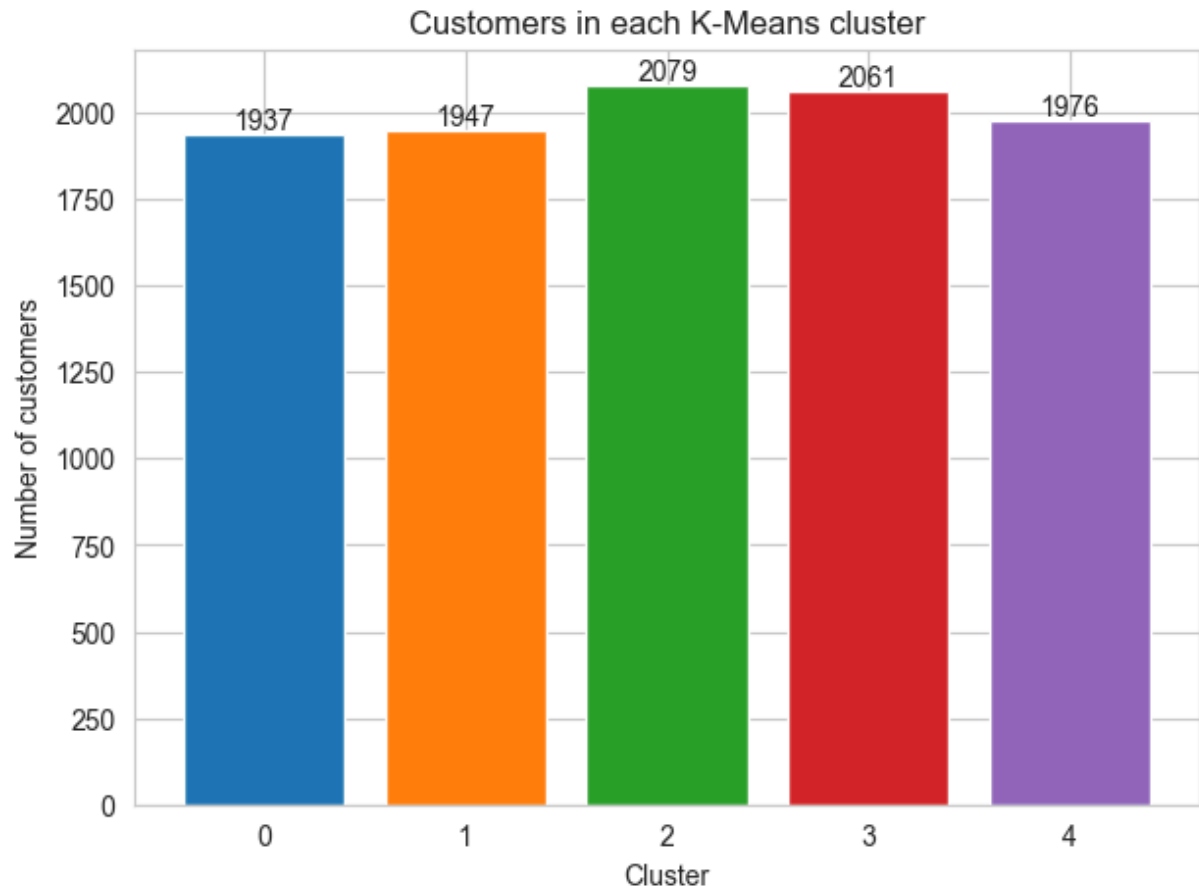
Selected K : 5
 Silhouette score : 0.0631
 Inertia : 438308.29

Cluster
 0 1937
 1 1947
 2 2079
 3 2061
 4 1976
 Name: count, dtype: int64

```
In [32]: # Cluster visualisation in PCA space
plt.figure(figsize=(9, 6))
for c in range(best_k):
    plt.scatter(X_pca[kmeans_labels == c, 0], X_pca[kmeans_labels == c, 1], s=12, a
centers_pca = pca.transform(final_kmeans.cluster_centers_)
plt.scatter(centers_pca[:, 0], centers_pca[:, 1], marker="X", s=250, c="black", lab
plt.xlabel("Principal Component 1"); plt.ylabel("Principal Component 2")
plt.title(f"K-Means Clusters (K={best_k}) using PCA")
plt.legend()
plt.show()
```



```
In [33]: counts = df_cluster["Cluster"].value_counts().sort_index()
plt.figure(figsize=(7, 5))
bars = plt.bar(counts.index.astype(str), counts.values, color=sns.color_palette("tab10"))
for b in bars:
    plt.text(b.get_x() + b.get_width() / 2, b.get_height(), int(b.get_height()), ha="center")
plt.xlabel("Cluster"); plt.ylabel("Number of customers"); plt.title("Customers in each cluster")
plt.show()
```



Cluster profiling

```
In [34]: profile_cols = ["Customer_Age", "Annual_Income", "Tenure_Years", "Family_Size", "La  
         "Spent_Wine", "Spent_Fruits", "Spent_Meat", "Spent_Fish", "Spent_Sw  
         "Total_Amount_Spent", "Deals_Purchased", "Web_Purchases", "Catalog_  
         "Store_Purchases", "Monthly_Web_Visits", "Campaigns_Accepted", "Cam  
  
cluster_profile = df_cluster.groupby("Cluster")[profile_cols].mean()  
cluster_profile.round(2).T
```

Out[34]:

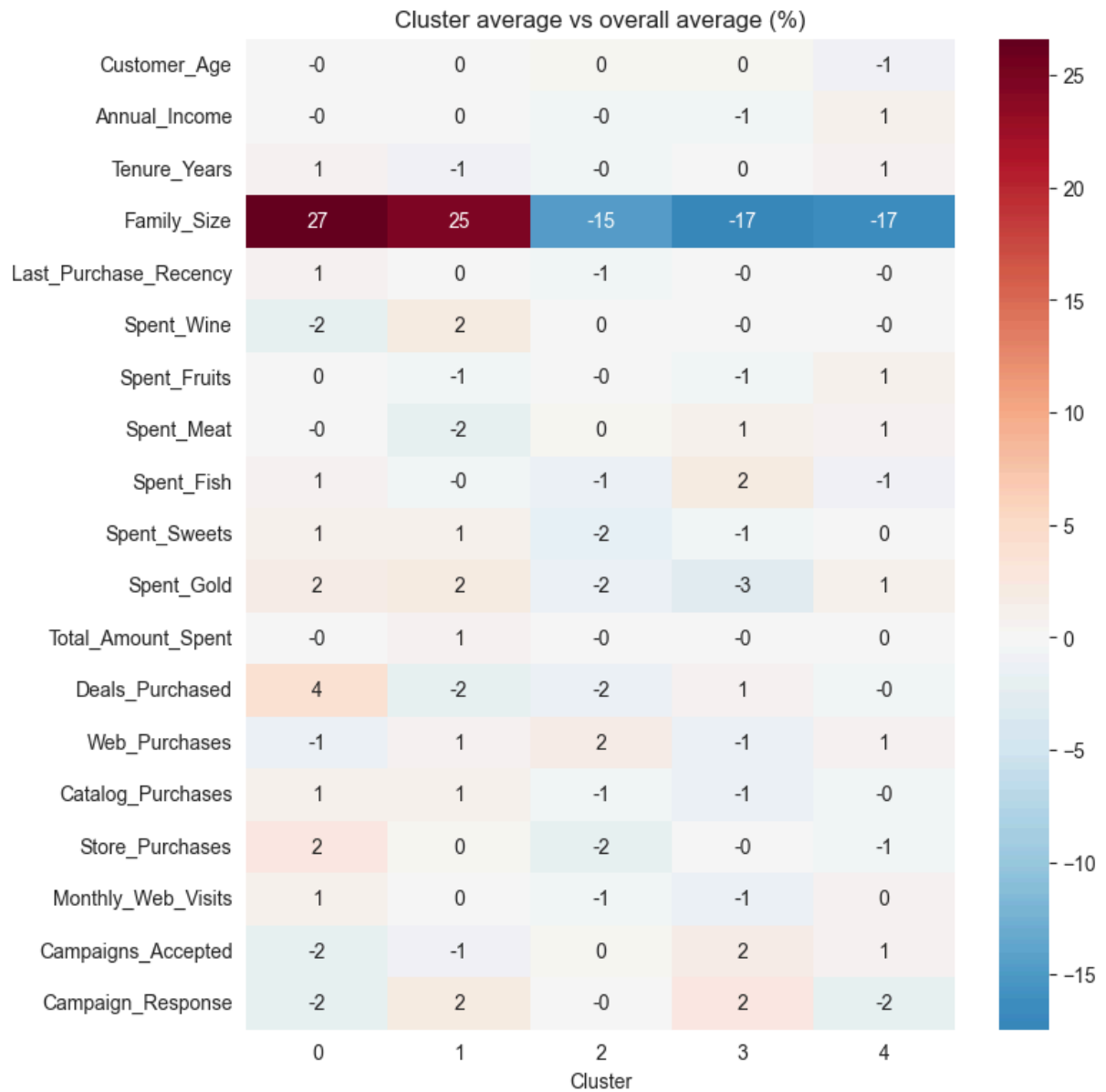
Cluster	0	1	2	3	4
Customer_Age	54.11	54.29	54.43	54.44	53.71
Annual_Income	49786.80	50083.99	49751.83	49638.69	50564.55
Tenure_Years	3.03	2.99	3.00	3.02	3.03
Family_Size	3.01	2.96	2.03	1.96	1.98
Last_Purchase_Recency	49.98	49.66	49.31	49.51	49.51
Spent_Wine	487.39	509.50	498.79	497.49	496.41
Spent_Fruits	49.85	49.46	49.68	49.45	50.12
Spent_Meat	400.66	393.98	402.49	405.15	403.84
Spent_Fish	149.60	148.20	147.17	151.64	147.38
Spent_Sweets	100.63	100.51	97.60	98.63	99.39
Spent_Gold	254.53	255.66	246.92	244.38	252.74
Total_Amount_Spent	1442.66	1457.30	1442.65	1446.75	1449.89
Deals_Purchased	4.68	4.41	4.44	4.54	4.49
Web_Purchases	4.42	4.51	4.56	4.42	4.51
Catalog_Purchases	4.51	4.51	4.43	4.39	4.44
Store_Purchases	4.63	4.54	4.43	4.52	4.49
Monthly_Web_Visits	9.61	9.49	9.42	9.34	9.52
Campaigns_Accepted	2.47	2.49	2.53	2.55	2.53
Campaign_Response	0.49	0.51	0.50	0.51	0.49

```

In [35]: # How far each cluster is from the overall average (in %), as a heatmap
overall_mean = df_cluster[profile_cols].mean()
diff_pct = (cluster_profile - overall_mean) / overall_mean * 100

plt.figure(figsize=(8, 9))
sns.heatmap(diff_pct.T, annot=True, fmt=".0f", cmap="RdBu_r", center=0)
plt.title("Cluster average vs overall average (%)")
plt.show()

```



```
In [36]: # Categorical make-up of every cluster (row %)
for col in ["Membership_Tier", "Education_Level", "Marital_Status", "Country", "Dev
print(f"\n--- {col} by Cluster (%) ---")
print((pd.crosstab(df_cluster["Cluster"], df_cluster[col], normalize="index")) *
```

--- Membership_Tier by Cluster (%) ---

Membership_Tier	Basic	Gold	Platinum	Silver
Cluster				
0	40.8	20.0	10.1	29.0
1	40.4	19.6	10.9	29.1
2	37.0	22.3	9.5	31.2
3	41.4	19.5	10.5	28.6
4	40.7	18.8	9.6	30.9

--- Education_Level by Cluster (%) ---

Education_Level	Basic	Graduation	Master	PhD
Cluster				
0	19.6	50.1	20.6	9.7
1	20.7	50.3	19.7	9.2
2	18.6	50.7	20.8	9.9
3	20.1	49.7	20.0	10.1
4	19.9	50.4	20.1	9.6

--- Marital_Status by Cluster (%) ---

Marital_Status	Divorced	Married	Single	Together	Widow
Cluster					
0	0.0	0.0	0.0	100.0	0.0
1	0.0	100.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	100.0
3	100.0	0.0	0.0	0.0	0.0
4	0.0	0.0	100.0	0.0	0.0

--- Country by Cluster (%) ---

Country	Brazil	Germany	India	UK	USA
Cluster					
0	20.4	21.2	20.1	19.4	18.8
1	20.4	20.9	18.4	20.0	20.3
2	21.3	18.7	21.9	18.9	19.2
3	22.6	20.4	19.3	17.7	19.9
4	20.2	21.3	18.2	21.9	18.4

--- Device_Type by Cluster (%) ---

Device_Type	Desktop	Mobile	Tablet
Cluster			
0	40.5	49.4	10.1
1	41.1	49.6	9.3
2	40.2	50.9	8.9
3	41.2	48.2	10.6
4	41.6	48.2	10.2

```
In [37]: print(pd.crosstab(df_cluster["Cluster"], df_cluster["Campaign_Response"], normalize=
```

Campaign_Response	0	1
Cluster		
0	50.9	49.1
1	48.7	51.3
2	49.8	50.2
3	48.5	51.5
4	50.9	49.1

Name the segments

Names are assigned **after** clustering, from the cluster profiles above (they are labels for interpretation, not inputs to the model). The profile tables show that the clusters are separated almost entirely by `Marital_Status` (and the family size that goes with it), so the names are built from the dominant marital status of each cluster.

```
In [38]: dominant_status = (
    df_cluster.groupby("Cluster")["Marital_Status"]
    .agg(lambda s: s.value_counts().index[0])
)
share = (
    df_cluster.groupby("Cluster")["Marital_Status"]
    .agg(lambda s: s.value_counts(normalize=True).iloc[0] * 100)
)
cluster_names = {c: f"{dominant_status[c]} customers" for c in dominant_status.index}

df_cluster["KMeans_Segment"] = df_cluster["Cluster"].map(cluster_names)
pd.DataFrame({
    "Segment": dominant_status.map(lambda x: f"{x} customers"),
    "Dominant_%": share.round(1),
    "Customers": df_cluster["Cluster"].value_counts().sort_index(),
    "Avg_Family_Size": cluster_profile["Family_Size"].round(2)})
```

Out[38]:

	Segment	Dominant_%	Customers	Avg_Family_Size
--	---------	------------	-----------	-----------------

Cluster

0	Together customers	100.0	1937	3.01
1	Married customers	100.0	1947	2.96
2	Widow customers	100.0	2079	2.03
3	Divorced customers	100.0	2061	1.96
4	Single customers	100.0	1976	1.98

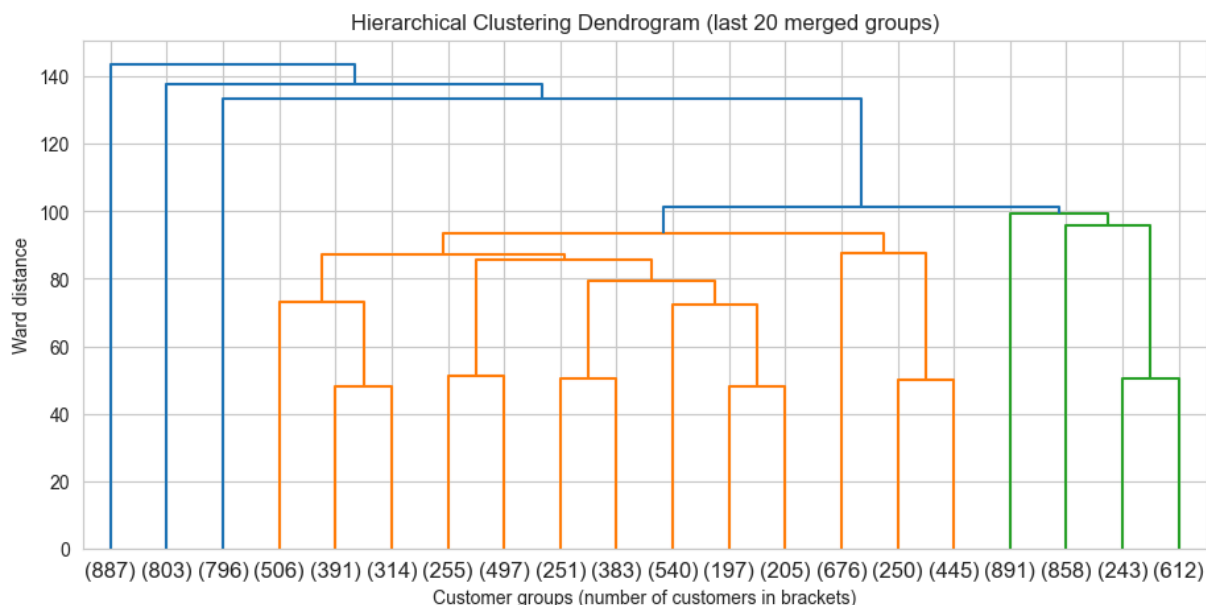
The only structure K-Means picks up is the categorical `Marital_Status` column. Treat the segments as a *starting point* (and be careful about business conclusions), not as proof of distinct customer types.

Hierarchical clustering (Agglomerative)

Dendrogram

```
In [39]: Z = linkage(X_scaled, method="ward")

plt.figure(figsize=(11, 5))
dendrogram(Z, truncate_mode="lastp", p=20)
plt.title("Hierarchical Clustering Dendrogram (last 20 merged groups)")
plt.xlabel("Customer groups (number of customers in brackets)")
plt.ylabel("Ward distance")
plt.show()
```



Choosing the number of clusters (same K range as K-Means, 2-10)

```
In [40]: from scipy.cluster.hierarchy import fcluster

hc_results = []
for k in range(2, 11):
    labels = fcluster(Z, t=k, criterion="maxclust") - 1  # cut the Ward tree in
    hc_results.append({"K": k, "Silhouette_Score": silhouette_score(X_scaled, label
hc_results = pd.DataFrame(hc_results)

best_k_hc = int(hc_results.loc[hc_results["Silhouette_Score"].idxmax(), "K"])
print(hc_results)
print("\nBest K:", best_k_hc, " | Highest silhouette:", round(hc_results["Silhouette
```

	K	Silhouette_Score
0	2	0.048919
1	3	0.052249
2	4	0.049840
3	5	0.026375
4	6	0.023416
5	7	0.019027
6	8	0.018244
7	9	0.015446
8	10	0.015547

Best K: 3 | Highest silhouette: 0.0522

```
In [41]: hc = AgglomerativeClustering(n_clusters=best_k_hc, linkage="ward")
df_cluster["Hierarchical_Cluster"] = hc.fit_predict(X_scaled)

print(df_cluster["Hierarchical_Cluster"].value_counts().sort_index())
df_cluster.groupby("Hierarchical_Cluster")[profile_cols].mean().round(2).T
```

```
Hierarchical_Cluster
0      8310
1      887
2      803
Name: count, dtype: int64
```

Out[41]:

Hierarchical_Cluster	0	1	2
Customer_Age	54.10	54.48	54.90
Annual_Income	50031.46	50306.45	48844.69
Tenure_Years	3.01	2.97	3.13
Family_Size	2.37	2.38	2.38
Last_Purchase_Recency	49.59	48.25	51.13
Spent_Wine	500.37	490.30	481.04
Spent_Fruits	49.83	48.97	49.26
Spent_Meat	401.58	396.86	403.24
Spent_Fish	148.67	145.93	153.35
Spent_Sweets	99.01	101.48	100.19
Spent_Gold	251.08	246.95	251.16
Total_Amount_Spent	1450.55	1430.50	1438.22
Deals_Purchased	4.52	4.49	4.42
Web_Purchases	4.48	4.58	4.43
Catalog_Purchases	4.46	4.41	4.44
Store_Purchases	4.53	4.45	4.47
Monthly_Web_Visits	9.48	9.46	9.42
Campaigns_Accepted	2.51	2.45	2.63
Campaign_Response	0.50	0.51	0.47

```
In [42]: print("Hierarchical clusters vs Marital_Status (row %):")
(pd.crosstab(df_cluster["Hierarchical_Cluster"], df_cluster["Marital_Status"], norm
```

Hierarchical clusters vs Marital_Status (row %):

Out[42]:

Marital_Status	Divorced	Married	Single	Together	Widow
Hierarchical_Cluster					
0	20.3	19.7	19.7	19.5	20.8
1	22.0	18.6	18.2	20.0	21.3
2	22.5	18.3	21.7	17.7	19.8

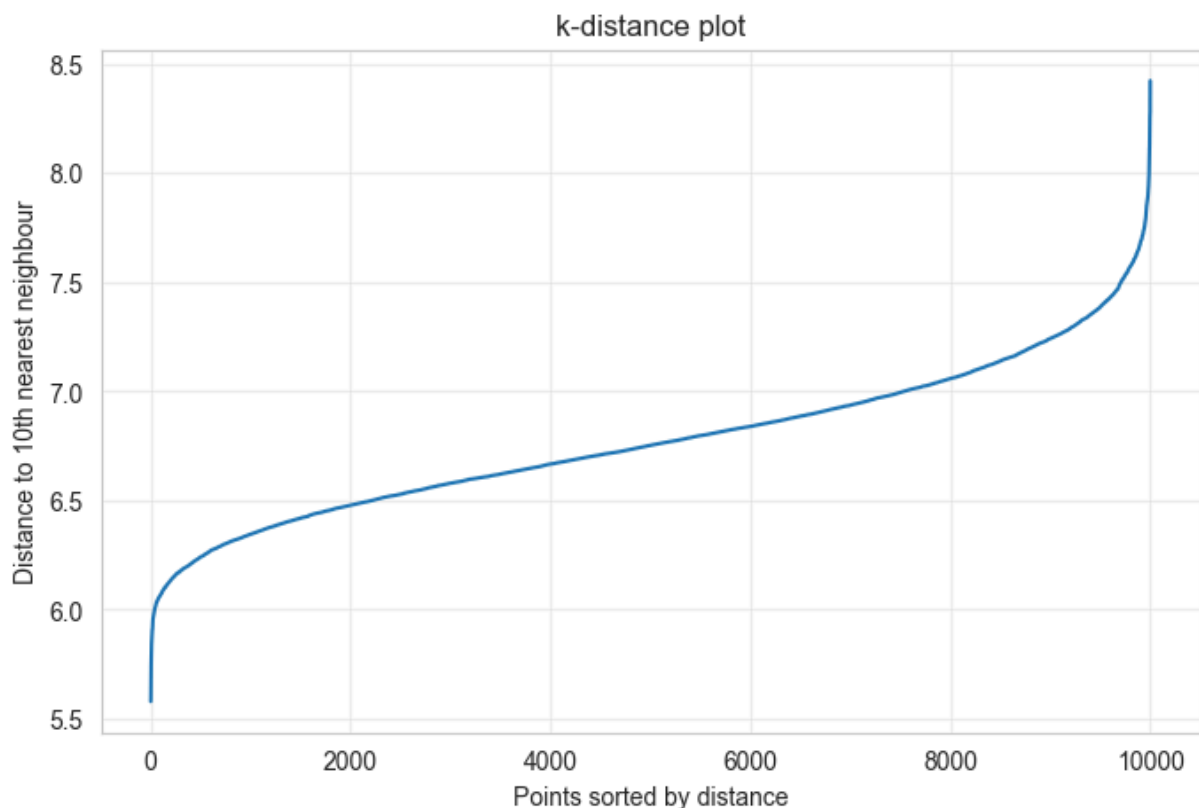
DBSCAN

k-distance plot to pick a sensible eps

```
In [43]: k_nn = 10
nn = NearestNeighbors(n_neighbors=k_nn).fit(X_scaled)
distances, _ = nn.kneighbors(X_scaled)
kdist = np.sort(distances[:, -1])

plt.figure(figsize=(8, 5))
plt.plot(kdist)
plt.xlabel("Points sorted by distance"); plt.ylabel(f"Distance to {k_nn}th nearest")
plt.title("k-distance plot")
plt.grid(alpha=0.3)
plt.show()

eps_candidates = np.round(np.percentile(kdist, [50, 70, 85, 95, 99]), 2)
print("eps candidates:", eps_candidates)
```



eps candidates: [6.75 6.94 7.14 7.39 7.68]

Grid search over eps and min_samples

```
In [44]: def summarise_labels(X, labels):
# Noise points (-1) are excluded from the scores, same as in the original notebook
mask = labels != -1
n_clusters = len(set(labels[mask]))
```

```

sizes = pd.Series(labels[mask]).value_counts()
out = {"clusters": n_clusters,
      "big_clusters": int((sizes >= 0.01 * len(labels)).sum()), # clusters h
      "noise_points": int((~mask).sum()),
      "noise_percentage": round((~mask).mean() * 100, 2)}
if n_clusters >= 2:
    out["silhouette_score"] = round(silhouette_score(X[mask], labels[mask]), 6)
else:
    out["silhouette_score"] = np.nan
return out

results_dbscan = []
for eps in eps_candidates:
    for min_samples in [5, 10, 20, 50]:
        labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X_scaled)
        results_dbscan.append({"eps": eps, "min_samples": min_samples, **summarise_

dbscan_results = pd.DataFrame(results_dbscan)
dbscan_results

```

Out[44]:

	eps	min_samples	clusters	big_clusters	noise_points	noise_percentage	silhouette_scc
0	6.75	5	9	1	594	5.94	-0.0469
1	6.75	10	1	1	1346	13.46	N
2	6.75	20	1	1	2794	27.94	N
3	6.75	50	2	1	7417	74.17	0.0074
4	6.94	5	1	1	220	2.20	N
5	6.94	10	1	1	478	4.78	N
6	6.94	20	1	1	1116	11.16	N
7	6.94	50	1	1	3641	36.41	N
8	7.14	5	1	1	53	0.53	N
9	7.14	10	1	1	125	1.25	N
10	7.14	20	1	1	335	3.35	N
11	7.14	50	1	1	1261	12.61	N
12	7.39	5	1	1	11	0.11	N
13	7.39	10	1	1	15	0.15	N
14	7.39	20	1	1	52	0.52	N
15	7.39	50	1	1	236	2.36	N
16	7.68	5	1	1	3	0.03	N
17	7.68	10	1	1	3	0.03	N
18	7.68	20	1	1	5	0.05	N
19	7.68	50	1	1	28	0.28	N



Selection rule: a DBSCAN result is only useful if it has **at least 2 real clusters** (each holding $\geq 1\%$ of customers) and **$\leq 20\%$ noise**. Tiny clusters of 2-5 customers are not segments, so they do not count.

```
In [45]: candidates = dbscan_results[(dbscan_results["big_clusters"] >= 2) & (dbscan_results["noise_percentage"] <= 20)]
print("Configurations that satisfy the rule:", len(candidates))
candidates.sort_values("silhouette_score", ascending=False).head(10)
```

Configurations that satisfy the rule: 0

```
Out[45]:
```

	eps	min_samples	clusters	big_clusters	noise_points	noise_percentage	silhouette_score
--	-----	-------------	----------	--------------	--------------	------------------	------------------



```
In [46]: if len(candidates) > 0:
best_row = candidates.sort_values("silhouette_score", ascending=False).iloc[0]
```

```

dbscan_note = "Best configuration among those with >=2 real clusters and <=20%
else:
    # No usable split exists -> use the configuration with the least noise to show
    fallback = dbscan_results[dbscan_results["noise_percentage"] <= 20]
    best_row = fallback.sort_values(["big_clusters", "noise_percentage"], ascending
    dbscan_note = ("No configuration gave >=2 real clusters with <=20% noise. DBSCAN
                  "group, so the lowest-noise configuration is kept for the compar

print(dbscan_note)
print(best_row.to_dict())

dbscan = DBSCAN(eps=float(best_row["eps"]), min_samples=int(best_row["min_samples"])
df_cluster["DBSCAN_Cluster"] = dbscan.fit_predict(X_scaled)
dbscan_counts = df_cluster["DBSCAN_Cluster"].value_counts().sort_index()
print()
print(dbscan_counts)

```

No configuration gave >=2 real clusters with <=20% noise. DBSCAN sees the data as one dense group, so the lowest-noise configuration is kept for the comparison.

```
{'eps': 7.68, 'min_samples': 5.0, 'clusters': 1.0, 'big_clusters': 1.0, 'noise_points': 3.0, 'noise_percentage': 0.03, 'silhouette_score': nan}
```

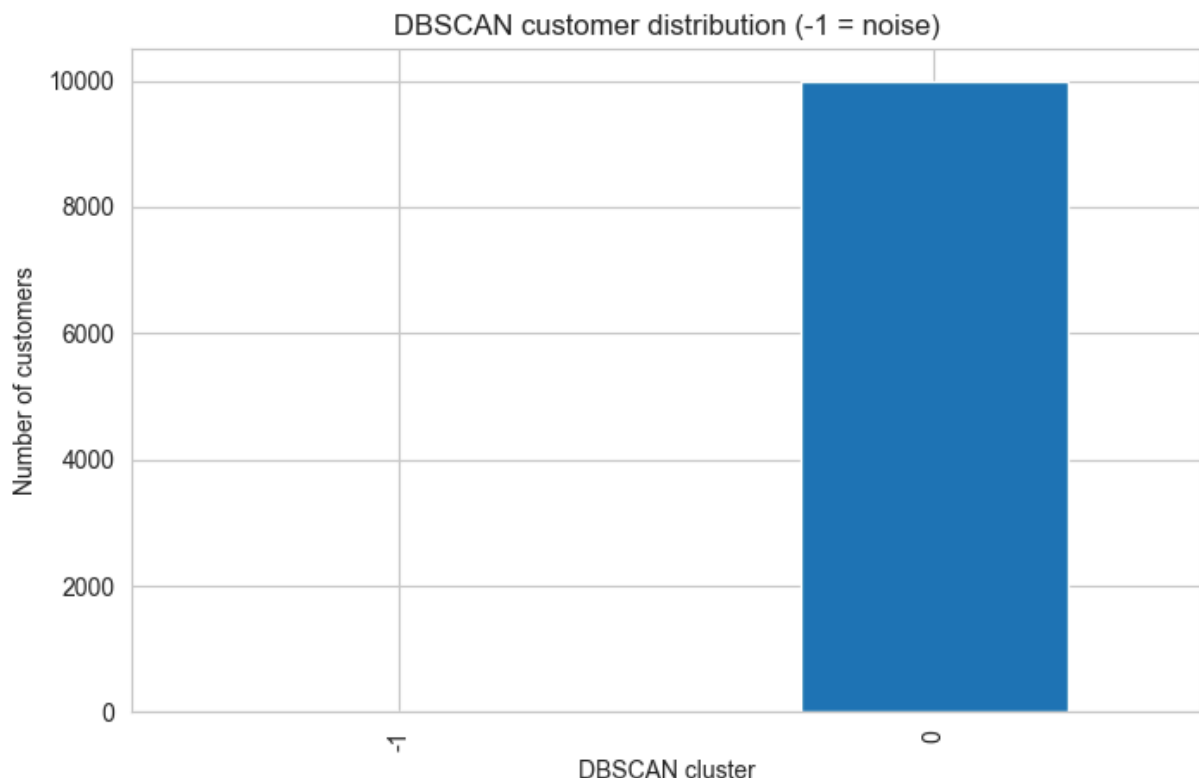
DBSCAN_Cluster

```
-1      3
```

```
0    9997
```

Name: count, dtype: int64

```
In [47]: dbscan_counts.plot(kind="bar", figsize=(8, 5), title="DBSCAN customer distribution
plt.xlabel("DBSCAN cluster"); plt.ylabel("Number of customers")
plt.show()
```



Reading the result: in 49 standardised dimensions almost every point has a similar distance to its neighbours, so a single `eps` either merges everything into one cluster or marks most customers as noise. DBSCAN is the weakest fit for this dataset, which is expected for evenly spread data without dense regions.

Compare all algorithms

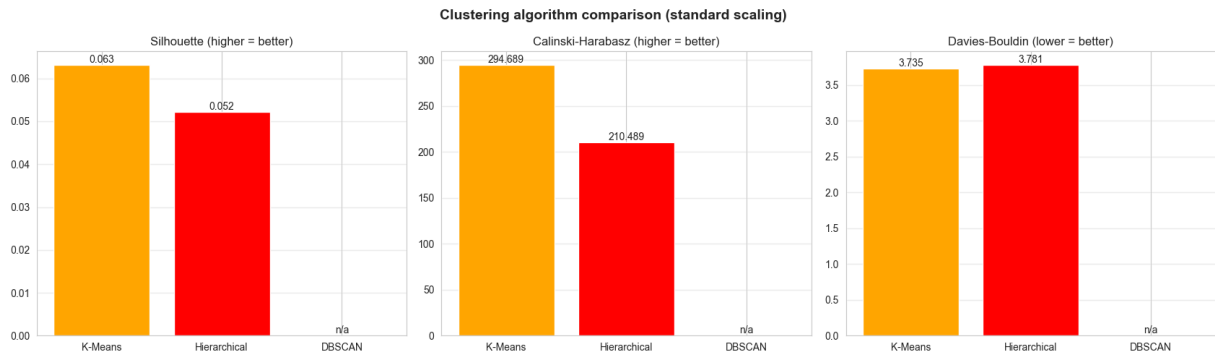
```
In [48]: def evaluate(name, labels):
    labels = np.asarray(labels)
    mask = labels != -1
    n_clusters = len(set(labels[mask]))
    row = {"Algorithm": name, "Clusters": n_clusters, "Noise_%": round((~mask).mean())}
    if n_clusters >= 2:
        row["Silhouette_Score"] = silhouette_score(X_scaled[mask], labels[mask])
        row["Calinski_Harabasz"] = calinski_harabasz_score(X_scaled[mask], labels[mask])
        row["Davies_Bouldin"] = davies_bouldin_score(X_scaled[mask], labels[mask])
    else:
        row["Silhouette_Score"] = row["Calinski_Harabasz"] = row["Davies_Bouldin"] = 0
    return row

comparison_df = pd.DataFrame([
    evaluate("K-Means", df_cluster["Cluster"]),
    evaluate("Hierarchical", df_cluster["Hierarchical_Cluster"]),
    evaluate("DBSCAN", df_cluster["DBSCAN_Cluster"]),
])
comparison_df.round(4)
```

```
Out[48]:
```

	Algorithm	Clusters	Noise_%	Silhouette_Score	Calinski_Harabasz	Davies_Bouldin
0	K-Means	5	0.00	0.0631	294.6891	3.7350
1	Hierarchical	3	0.00	0.0522	210.4892	3.7811
2	DBSCAN	1	0.03	NaN	NaN	NaN

```
In [49]: fig, axes = plt.subplots(1, 3, figsize=(17, 5))
    palette = ["orange", "red", "green", "purple"]
    for ax, metric, title in zip(axes,
        ["Silhouette_Score", "Calinski_Harabasz", "Davies_Bouldin"],
        ["Silhouette (higher = better)", "Calinski-Harabasz (higher = better)", "Davies-Bouldin (lower = better)"]):
        bars = ax.bar(comparison_df["Algorithm"], comparison_df[metric].fillna(0), color=palette)
        ax.set_title(title); ax.grid(axis="y", alpha=0.3)
        for b, v in zip(bars, comparison_df[metric]):
            ax.text(b.get_x() + b.get_width() / 2, b.get_height(), "n/a" if np.isnan(v) else v,
                ha="center", va="bottom")
    plt.suptitle("Clustering algorithm comparison (standard scaling)", fontsize=14, fontweight="bold")
    plt.tight_layout()
    plt.show()
```



13. Final segmentation and save results

```
In [50]: final_choice = comparison_df.loc[comparison_df["Silhouette_Score"].idxmax(), "Algorithm"]
print("Algorithm with the highest silhouette score:", final_choice)

final_col = label_cols[final_choice]
df_cluster["Final_Cluster"] = df_cluster[final_col]
final_profile = df_cluster.groupby("Final_Cluster")[profile_cols].mean().round(2)
final_profile.T
```

Algorithm with the highest silhouette score: K-Means

```
-----
NameError                                Traceback (most recent call last)
Cell In[50], line 4
      1 final_choice = comparison_df.loc[comparison_df["Silhouette_Score"].idxmax(),
      2 "Algorithm"]
      3 print("Algorithm with the highest silhouette score:", final_choice)
----> 4 final_col = label_cols[final_choice]
      5 df_cluster["Final_Cluster"] = df_cluster[final_col]
      6 final_profile = df_cluster.groupby("Final_Cluster")[profile_cols].mean().rou
nd(2)
      7 final_profile.T

NameError: name 'label_cols' is not defined
```

```
In [ ]: df_cluster.to_csv("customer_data_clustered.csv", index=False)
print("Saved customer_data_clustered.csv", df_cluster.shape)
df_cluster[["Customer_ID", "Cluster", "KMeans_Segment", "Hierarchical_Cluster", "DBSCAN_Segment"]]
```

Saved customer_data_clustered.csv (10000, 52)

Out[]:

	Customer_ID	Cluster	KMeans_Segment	Hierarchical_Cluster	DBSCAN_Cluster
0	1	2	Widow customers	0	0
1	2	2	Widow customers	1	0
2	3	1	Married customers	1	0
3	4	2	Widow customers	0	0
4	5	1	Married customers	0	0
5	6	0	Together customers	0	0
6	7	2	Widow customers	0	0
7	8	0	Together customers	0	0
8	9	2	Widow customers	1	0
9	10	0	Together customers	0	0

14. Summary and conclusions

Results on this dataset

Algorithm	Chosen setting	Silhouette	Comment
K-Means	K = 5	0.063 (best)	clusters = the five <code>Marital_Status</code> groups
Hierarchical (Ward)	K = 3	0.052	one big group (~83%) plus two small groups
DBSCAN	-	n/a	no usable split; the data forms one dense group

Key takeaways

1. **K-Means is the best of the four** by silhouette, Calinski-Harabasz and Davies-Bouldin among the algorithms that produced a real segmentation, but its score (~0.06) is still very low, i.e. the clusters overlap heavily.
2. The K-Means clusters are simply the **marital-status groups**; within them, spending, income, age, purchase channels and campaign response are almost identical. `Together` and `Married` customers have larger households (about 3 people versus 2).